

Designing Verification Checkpoints to Mitigate Silent Errors in Agentic AI Pipelines

Jahanvi Mishra

Indian Institute of Information Technology Allahabad

Prayagraj, India

mns2026004@iiita.ac.in

Abstract

Agentic AI systems increasingly combine large language models with tools, external environments, memory, planning, and multi-step reasoning to perform complex tasks autonomously. While these systems can solve tasks beyond the capabilities of standalone language models, their multi-step nature introduces a distinctive reliability problem: silent errors. A silent error is an incorrect intermediate decision, action, or state transition that is not detected and subsequently propagates through the agentic pipeline, potentially producing an incorrect final outcome. Unlike explicit failures, silent errors may not trigger exceptions, warnings, or other visible signals, making them difficult to detect and recover from. This paper examines verification checkpoints as a mechanism for detecting and containing such errors before they propagate. It reviews related work on agentic systems, process supervision, hallucination detection, retrieval-grounded verification, self-consistency, and evaluation of LLM-based agents. Building on these approaches, the paper proposes a conceptual verification-checkpoint framework that places explicit validation stages at high-risk points in an agentic workflow. Each checkpoint evaluates the validity of the current state, action, observation, or outcome and can trigger actions such as PASS, REPAIR, RETRY, ESCALATE, or ABORT. The framework emphasizes risk-adaptive checkpoint placement, independent verification, deterministic validation where possible, and explicit recovery mechanisms. The paper also identifies research challenges including verifier failure, correlated errors, verification cost, non-deterministic environments, and the difficulty of defining ground truth. Finally, several research directions are outlined, including standardized silent-error benchmarks, causal error-trace datasets, adaptive checkpoint scheduling, fault-injection testing, adversarial verification, and calibrated human intervention.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**; • **Information systems** → *Information retrieval*; • **Software and its engineering** → *Software verification and validation*.

Keywords

agentic AI, silent errors, verification checkpoints, LLM agents, reliability, error propagation, process supervision, AI safety

1 Introduction

Large language models (LLMs) have evolved from systems that generate text into agents capable of reasoning, planning, using external tools, interacting with environments, and coordinating multiple steps to achieve a goal. Frameworks such as ReAct demonstrate how language models can interleave reasoning and actions

[1], while Toolformer demonstrates the ability of language models to learn when and how to invoke external tools [2]. Multi-agent frameworks such as AutoGen further extend these capabilities by allowing multiple language-model-based agents to communicate and collaborate [3]. These developments have enabled increasingly sophisticated agentic systems that can operate across extended sequences of decisions and actions.

However, increased autonomy also introduces new reliability challenges. In a traditional software system, incorrect behavior may result in explicit errors, exceptions, or failed tests. Agentic AI systems can instead produce incorrect intermediate reasoning, invalid tool calls, unsupported assumptions, or erroneous state transitions while continuing execution normally. Such failures may remain undetected because the system itself does not necessarily know that an intermediate result is incorrect.

This problem becomes particularly important in long-horizon agentic workflows. An early error can influence subsequent reasoning and actions, causing later steps to build on an incorrect state. As the number of dependent steps increases, the possibility of error propagation also increases. Evaluation benchmarks such as AgentBench and SWE-bench demonstrate the complexity of evaluating agents across interactive and real-world tasks [4, 5]. Recent surveys also highlight the need for systematic evaluation methods for LLM-based agents [6, 7].

The central problem addressed in this paper is therefore the detection and containment of silent errors in agentic AI pipelines. Rather than relying only on final-output evaluation, verification should be introduced at intermediate points in the execution process. These verification stages are referred to as *verification checkpoints*.

The remainder of this paper examines the nature of silent errors, develops a conceptual checkpoint architecture, discusses checkpoint placement and recovery, reviews current approaches, and proposes a verification-checkpoint framework. The research challenges and future directions are then discussed before the paper concludes.

2 Background and Related Work

2.1 From language models to agentic systems

Modern language models increasingly operate as components within larger agentic systems. ReAct introduced an approach in which reasoning and acting are interleaved, enabling models to use external information and tools during task execution [1]. Toolformer explored the use of external tools by training language models to determine when tool calls are useful [2]. These approaches demonstrate that an LLM can serve not only as a text generator but also as a decision-making component.

Multi-agent systems extend this idea by coordinating multiple agents or roles. AutoGen provides a framework for building applications based on multi-agent conversation [3]. Such systems can perform complex tasks by dividing responsibilities among agents, but the additional interactions also introduce more opportunities for incorrect information and decisions to propagate.

2.2 Reasoning and process supervision

Chain-of-thought prompting demonstrated that explicitly eliciting intermediate reasoning steps can improve performance on reasoning tasks [8]. Self-consistency further improves reasoning by sampling multiple reasoning paths and selecting a consistent answer [9].

Process supervision focuses on evaluating intermediate reasoning rather than only evaluating the final answer. Lightman et al. investigated step-by-step verification of mathematical reasoning, showing the potential value of checking intermediate steps [10]. This idea is closely related to verification checkpoints because both approaches seek to identify errors before they influence the final outcome.

2.3 Hallucination and factual verification

Hallucination detection has become an important component of reliable LLM systems. SelfCheckGPT investigates hallucination detection by comparing multiple samples from a model without requiring external knowledge [11]. FActScore evaluates the factual precision of long-form generated text using atomic facts [12].

Retrieval-grounded approaches can also provide external evidence for verification. RAGAs proposes automated evaluation methods for retrieval augmented generation systems [13]. These approaches demonstrate that verification can be performed using information external to the model’s own generation process.

2.4 Evaluation of LLM agents

Agent evaluation differs from ordinary language-model evaluation because agents must interact with environments, tools, and multi-step tasks. AgentBench provides a benchmark for evaluating LLMs as agents across multiple environments [4]. SWE-bench evaluates the ability of language models to resolve real-world GitHub issues [5].

Recent surveys emphasize the importance of systematic evaluation methodologies for LLM-based agents [6, 7]. However, evaluation alone does not necessarily prevent errors during execution. This motivates the need for mechanisms that verify intermediate states while an agent is running.

LLM-as-a-judge methods provide a scalable way to evaluate open-ended agent outputs. Zheng et al. found that strong LLM judges can align reasonably well with human preferences, while also revealing systematic biases such as position and verbosity effects [14]. These findings are directly relevant to automated checkpoints, since they show that a verifier is itself an imperfect model and should not be treated as an infallible oracle.

3 Silent Errors in Agentic AI Pipelines

3.1 Definition

A silent error is an incorrect intermediate decision, action, observation, assumption, or state transition that is not detected by the system and therefore continues through the execution pipeline. Unlike explicit failures, silent errors do not necessarily generate an exception, warning, or other observable failure signal.

In an agentic system, a silent error can occur when an agent incorrectly interprets a user request, selects an inappropriate tool, supplies incorrect arguments, misinterprets a tool result, or updates its internal state using incorrect information. Because subsequent steps may depend on this state, the initial error can influence the remainder of the execution.

3.2 Error propagation

In a multi-step pipeline, the probability of an undetected error surviving multiple stages can be expressed conceptually as

$$P(\text{undetected error}) = 1 - \prod_i (1 - p_i(1 - q_i)) \quad (1)$$

where p_i represents the probability that an error occurs at stage i , and q_i represents the probability that the error is detected at that stage.

The equation illustrates that errors introduced at multiple stages can accumulate when verification is incomplete. A system that only evaluates the final output may therefore fail to detect an error introduced much earlier in the pipeline.

3.3 Sources of silent error

Silent errors can arise from several sources, including incorrect reasoning, hallucinated information, invalid tool selection, malformed tool arguments, incorrect interpretation of observations, stale memory, and failures in communication between multiple agents.

The risk becomes greater when later decisions depend strongly on earlier outputs. In such cases, an error introduced at one point can become embedded in the state of the system and subsequently appear to be valid information.

These source categories are not mutually exclusive in practice, but grouping them by origin helps identify which checkpoint layer is best positioned to catch each type of failure. The taxonomy of these sources is illustrated in Figure 1.

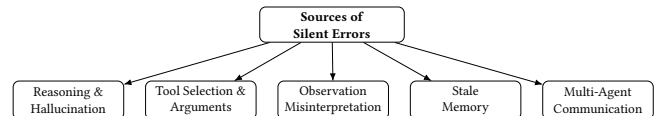


Figure 1: Taxonomy of silent-error sources in agentic AI pipelines.

4 Verification Checkpoints as a Reliability Mechanism

4.1 Conceptual model

A verification checkpoint is an explicit validation stage inserted into an agentic workflow. Instead of allowing every state transition to proceed automatically, the system evaluates whether the current state satisfies a verification predicate.

The checkpoint action can be represented as

$$C_i(S_i) \rightarrow \{\text{PASS, REPAIR, RETRY, ESCALATE, ABORT}\} \quad (2)$$

where S_i is the system state at checkpoint i .

A checkpoint therefore transforms a simple pipeline

$$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3$$

into a controlled process in which state transitions are validated before propagation.

The verification predicate can be represented as

$$V_i(S_i) = 1 \quad (3)$$

If $V_i(S_i) = 0$, propagation should be prevented until an appropriate recovery mechanism is executed.

4.2 Five-layer checkpoint architecture

A verification checkpoint can operate at multiple layers of an agentic execution.

Table 1: Five-layer verification checkpoint architecture

Layer	Verification Target	Example Verification
Input	User request and constraints	Verify task intent, required parameters, and ambiguous requirements.
Plan	Agent plan	Verify subgoals, dependencies, constraints, and action ordering.
Tool Call	Tool and arguments	Verify tool identity, argument types, values, permissions, and expected effects.
State	Intermediate state	Verify invariants, consistency with previous facts, and state transitions.
Output	Final result	Verify task satisfaction, evidence, safety constraints, and required format.

The five verification layers and their primary validation targets are summarized in Table 1.

The first layer verifies the interpretation of the user’s request. The second layer verifies the feasibility and safety of the agent’s plan. The third layer verifies tool selection, arguments, and permissions before and after invocation. The fourth layer verifies whether the current state remains consistent with previously established

facts and invariants. The fifth layer verifies the completed result against the original task, supporting evidence, and required constraints.

For plan verification, the predicate can be represented as

$$V_P(P, x) = 1 \quad (4)$$

where P represents a plan and x represents the task context.

Tool calls can also be checked before and after execution:

$$V_{\text{pre}}(\text{tool, args, context}) \quad (5)$$

$$V_{\text{post}}(\text{tool, args, result, context}) \quad (6)$$

Pre-verification determines whether a proposed action is valid before it is executed, while post-verification determines whether the resulting observation or state is consistent with expectations.

5 Designing Effective Checkpoints

5.1 Risk-adaptive checkpoint placement

Not every step in an agentic pipeline requires the same level of verification. Checkpoint placement should therefore consider the risk associated with a particular state transition.

The set of information relevant to a checkpoint can be represented as

$$I(S_t) = \{i_1, i_2, \dots, i_k\} \quad (7)$$

where $I(S_t)$ represents the information available at state S_t .

The risk associated with a potential error can be represented as

$$R_i = P(E_i) \times I_i \quad (8)$$

where $P(E_i)$ is the probability of error E_i and I_i represents its potential impact.

Checkpoint selection can then be expressed as

$$K_i = f(R_i, C_i, D_i) \quad (9)$$

where C_i represents verification cost and D_i represents dependency or downstream impact.

This produces a risk-adaptive architecture rather than a uniformly expensive one.

The resulting risk-adaptive decision process is illustrated in Figure 2.

5.2 Deterministic versus model-based verification

Where deterministic validation is possible, it can provide a strong basis for verification. Examples include schema validation, range checks, type checking, mathematical constraints, and consistency rules.

Model-based verification can be useful when deterministic rules are difficult to define. However, the verifier itself may produce incorrect judgments. A simple confidence-based condition can be expressed as

$$P(\text{correct}) < \tau \quad (10)$$

where τ is a minimum confidence threshold.

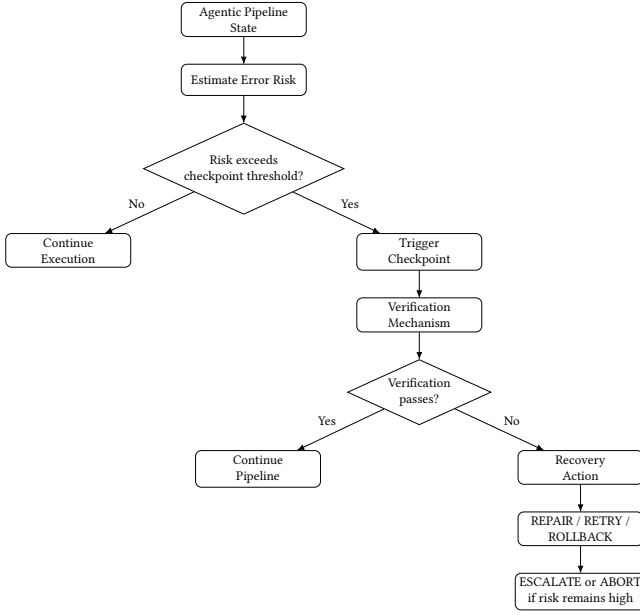


Figure 2: Risk-adaptive checkpoint decision and recovery flow.

5.3 Independent verification

Independent verification reduces the possibility that the same reasoning process produces both an error and its verification. Verification can be performed by a separate model, deterministic program, retrieval system, or human reviewer.

6 Error Recovery and Checkpoint Actions

A checkpoint is useful only if detection can lead to an appropriate recovery action. Depending on the type and severity of the detected error, the system may repair the state, retry the operation, roll back to an earlier valid state, escalate the decision to another component, or abort execution.

Rollback restores the last verified state, which can be represented as

$$S_i \rightarrow S_{i-k} \quad (11)$$

Rollback is valuable when an agent has performed reversible state-changing operations. Escalation transfers the decision to a human or a higher-assurance system when the estimated probability of correctness falls below an acceptable threshold, or when the consequence of an error exceeds a defined limit. This is especially important for high-impact actions where autonomous recovery may itself introduce additional risk.

7 Current Approaches and Developments

7.1 Process supervision

Process supervision evaluates intermediate reasoning steps instead of relying only on the final answer. This approach is particularly relevant to checkpoint design because intermediate errors can be detected before they propagate [10].

7.2 Self-consistency

Self-consistency samples multiple reasoning paths and selects an answer based on agreement among those paths [9]. It can provide an additional verification signal, although agreement between multiple outputs does not guarantee correctness.

7.3 Retrieval-grounded verification

Retrieval-grounded methods compare generated information against retrieved evidence. RAGAs provides evaluation measures for retrieval augmented generation systems [13]. External evidence can help reduce unsupported claims and provide a basis for verification.

7.4 LLM-as-a-judge

LLM-based judges can evaluate generated responses according to specified criteria. Zheng et al. studied the use of LLMs as judges and evaluated their agreement with human judgments [14]. However, model-based judges may share biases or failure modes with the systems they evaluate.

7.5 Red-team-based verification

Red-team approaches intentionally attempt to expose weaknesses in AI systems. Perez et al. investigated the use of language models for red teaming language models [15]. Such techniques can complement checkpoint mechanisms by testing whether verification controls can withstand adversarial or unexpected inputs.

7.6 Comparative summary of existing approaches

The approaches reviewed above target different stages of an agentic pipeline and rely on different verification mechanisms, ranging from deterministic checks to independent model-based judges. Table 2 summarizes these approaches according to their core idea, the checkpoint layer(s) from Section 4.2 they are best suited to, and their main limitation with respect to silent-error detection.

The reviewed studies use different experimental settings, including mathematical reasoning tasks, text-generation benchmarks, retrieval-augmented generation datasets, interactive agent environments, and software-engineering benchmarks. Their findings therefore provide complementary evidence rather than a directly comparable evaluation of silent-error detection.

The comparison in Table 2 shows that no single existing approach covers every checkpoint layer identified in Section 4.2: most methods concentrate on output-stage verification, while plan-, tool-call-, and state-level checks are comparatively underexplored. Several of the deterministic checks discussed in Section 5.2 complement the confidence-based condition in (10), and this gap in coverage motivates the layered, risk-adaptive framework proposed in Section 8.

8 Proposed Verification-Checkpoint Framework

The proposed framework organizes verification around the major components of an agentic execution: intent, planning, actions, observations, state, and outcomes. Rather than treating verification as a single final-stage operation, the framework introduces checkpoints at selected high-risk transitions.

Table 2: Comparative summary of existing verification- and evaluation-related approaches

Approach	Methodology / Key Finding	Applicable Layer(s)	Key Limitation
ReAct [1]	Interleaves reasoning and acting so the model can use tool feedback during execution	Plan, Tool-Call	Reasoning traces are not independently checked for correctness
Toolformer [2]	Trains a model to decide when and how to invoke external tools	Tool-Call	Tool-use decisions are learned, not explicitly verified at runtime
AutoGen [3]	Coordinates multiple agents and tools within one conversation-based workflow	Plan, State	Additional agent-to-agent channels create more silent-failure points
Chain-of-Thought prompting [8]	Elicits explicit intermediate reasoning steps	Input, Plan	Exposed reasoning is not guaranteed to be correct
Self-Consistency [9]	Samples multiple reasoning paths and selects the most consistent answer	Output	Agreement does not guarantee correctness if errors are correlated
Process supervision [10]	Evaluates intermediate reasoning steps rather than only the final answer	Plan, State	Demonstrated mainly on structured mathematical reasoning
SelfCheckGPT [11]	Detects hallucination by comparing multiple stochastic generations	Output	Requires multiple generations, increasing verification cost
FactScore [12]	Decomposes text into atomic facts and checks support from sources	Output	Depends on availability of reliable external sources
RAGAs [13]	Evaluates retrieval quality and faithfulness in retrieval-augmented systems	Tool-Call, Output	Limited to retrieval-augmented pipelines
LLM-as-a-judge [14]	Uses a separate LLM to evaluate open-ended outputs	Output	Judge model can share biases with the system it evaluates
Red-teaming [15]	Uses adversarial testing to expose weaknesses in model behavior	Tool-Call, State	Coverage depends on the adversarial cases considered

A checkpoint may therefore be associated with the task definition, a generated plan, a tool call, an observation, a state update, or the final result. Each checkpoint evaluates a verification predicate and selects an appropriate action based on the result.

The resulting VCP-AI architecture is illustrated in Figure 3.

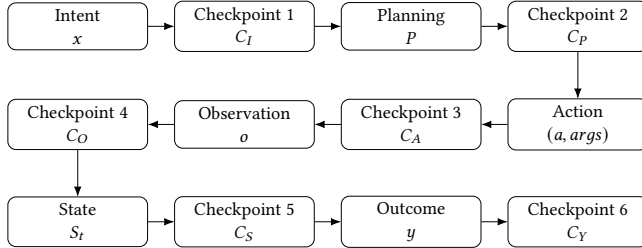


Figure 3: VCP-AI layered verification architecture for agentic AI pipelines.

8.1 Checkpoint effectiveness

The effectiveness of a verification mechanism can be evaluated using several metrics.

Detection recall can be represented as

$$DR = \frac{TP}{TP + FN} \quad (12)$$

where TP denotes true positives and FN denotes false negatives.

False-positive rate can be represented as

$$FPR = \frac{FP}{FP + TN} \quad (13)$$

where FP denotes false positives and TN denotes true negatives. Containment rate can be defined as

$$CR = \frac{\text{errors prevented from reaching final action}}{\text{errors introduced}} \quad (14)$$

Verification overhead can be represented as

$$VO = \frac{\text{additional latency or computational cost}}{\text{baseline cost}} \quad (15)$$

Finally, the silent-error rate can be expressed as

$$SER = \frac{\text{undetected harmful errors}}{\text{total execution opportunities}} \quad (16)$$

These measures allow verification mechanisms to be evaluated not only according to whether they detect errors, but also according to their cost and ability to prevent errors from reaching consequential actions.

9 Research Challenges and Limitations

9.1 The verifier can fail

A verification mechanism is itself a component of the system and can therefore produce incorrect judgments. A verifier may fail to detect an error, incorrectly reject a correct result, or misinterpret the state being evaluated.

9.2 Correlated errors

Independent verification is difficult when multiple verification components share the same model, training data, assumptions, or information sources. Correlated failures can therefore allow the same error to pass through multiple checkpoints.

9.3 Verification cost

Additional verification increases computational cost and latency. The total execution cost can be represented as

$$T_{\text{total}} = T_{\text{agent}} + \sum_i T_{C_i} \quad (17)$$

where T_{agent} is the original agent execution time and T_{C_i} represents the time associated with checkpoint i . Excessive checkpointing may reduce the efficiency and responsiveness of an agentic system. The challenge is therefore to identify checkpoints that provide meaningful reliability improvements without imposing unnecessary overhead.

9.4 Non-deterministic environments

Agentic systems may operate in environments where actions produce different results at different times. Verification must therefore account for uncertainty and distinguish genuine errors from valid environmental variation.

9.5 Difficulties in defining ground truth

Some agentic tasks do not have a single objectively correct intermediate state. In such cases, verification requires task-specific criteria, external evidence, or human judgment. Defining reliable ground truth is therefore an important research challenge.

10 Research Gaps and Future Directions

10.1 Standardized silent-error benchmarks

Existing agent benchmarks evaluate overall task performance, but systematic benchmarks specifically designed to measure silent-error detection and containment remain necessary. Such benchmarks should identify where errors are introduced, whether they are detected, and whether they reach the final action.

10.2 Causal error-trace datasets

Datasets containing complete execution traces could support research into the causal relationships between intermediate errors and final outcomes. Such traces can be represented as

$$\tau = (S_0, A_1, O_1, \dots, A_n, O_n, S_n) \quad (18)$$

where S_i represents a system state, A_i represents an action, and O_i represents an observation. Such datasets could annotate the first erroneous state, error type, detection point, propagation path, final consequence, and recovery outcome, enabling research into causal fault localization rather than final-answer classification alone.

10.3 Adaptive checkpoint scheduling

Future systems could dynamically determine when checkpoints are required based on estimated risk, task characteristics, state uncertainty, and downstream impact. A checkpoint can be triggered when the estimated risk exceeds the cost of verification:

$$\text{Checkpoint if } P(E_i) \times I_i > \lambda C_i \quad (19)$$

where λ controls the trade-off between risk and verification cost. The overall checkpoint-selection problem can then be formulated as

$$\text{minimize } \text{Cost}(C) + \lambda \times \text{ResidualRisk}(C) \quad (20)$$

where C denotes the selected set of checkpoints. This formalization could allow verification resources to be concentrated at high-risk points, and could lead to learned checkpoint schedulers.

10.4 Fault-injection testing for agents

Fault injection can intentionally introduce errors into agentic execution in order to evaluate whether checkpoint mechanisms detect and contain them. Such testing could provide a systematic way to measure robustness.

10.5 Verification under adversarial conditions

Verification mechanisms should also be evaluated against adversarial inputs and attempts to bypass safety controls. Red-team approaches can provide one method for identifying weaknesses in checkpoint systems.

10.6 Human intervention as a calibrated resource

Human review should be treated as a limited resource rather than a universal fallback. Future systems could escalate to a human only when

$$U(\text{autonomous action}) < U(\text{human review}) \quad (21)$$

where U represents the expected utility of the respective action, taking into account the probability of error, its consequence, latency, and review cost. This would transform human oversight from a generic fallback into a risk-calibrated verification mechanism.

11 Conclusion

Agentic AI systems introduce new reliability challenges because errors can occur at intermediate stages and propagate without producing explicit failures. Silent errors are particularly problematic because they may remain undetected until they influence a consequential final action.

Verification checkpoints provide a mechanism for detecting and containing such errors during execution rather than relying exclusively on final-output evaluation. The framework proposed in this paper organizes verification around intent, planning, actions, observations, state, and outcomes. It also emphasizes risk-adaptive checkpoint placement, independent verification, and explicit recovery actions.

The risk-adaptive placement discussed in Section 5.1 complements the layered architecture described in Section 4.2. The verification predicates described in Equation 3 and the checkpoint action model in Equation 2 provide formal representations of these controls.

The effectiveness of checkpoint mechanisms depends not only on their ability to detect errors but also on their cost, independence, robustness, and ability to trigger appropriate recovery actions. Important challenges include verifier failure, correlated errors, verification overhead, non-deterministic environments, and the absence of clearly defined ground truth.

Future research should therefore focus on standardized silent-error benchmarks, causal error-trace datasets, adaptive checkpoint scheduling, fault-injection testing, adversarial verification, and calibrated human intervention. These directions can help establish verification checkpoints as a systematic reliability mechanism for increasingly autonomous AI systems.

References

- [1] Shunyu Yao, Jeffrey Zhao, Dong Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *ICLR*, 2023.
- [2] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.
- [3] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Ahmed Hassan Awadallah, Ryen W. White, Daniel Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [4] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hanyu Ding, Kai Men, Ke Yang, Shun Zhang, Xin Deng, Aohan Zeng, Zheng Du, Sheng Zhang, Shimin Shen, Tian Zhang, Yuyu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents. *ICLR*, 2024.
- [5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.
- [6] M. Mohammadi, Y. Li, J. Lo, and W. Yip. Evaluation and benchmarking of llm agents: A survey. *arXiv preprint arXiv:2507.21504*, 2025.
- [7] Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. A survey on evaluation of llm-based agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 26690–26714, 2026.
- [8] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS 35*, pages 24824–24837, 2022.
- [9] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *ICLR*, 2023.
- [10] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- [11] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In *EMNLP*, pages 9004–9017, 2023.
- [12] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. Factscore: Fine-grained atomic evaluation of factual precision in long form text generation. In *EMNLP*, pages 12076–12100, 2023.
- [13] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation. In *EACL System Demonstrations*, pages 150–158, 2024.
- [14] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.
- [15] Ethan Perez, Saffron Huang, Francis Song, Tessa Cai, Remi Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. In *EMNLP*, 2022.
- [16] Nelson F. Liu, Kevin Lin, John Hewitt, Aditi Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [17] J. Kim, X. Liu, Z. Wang, S. Qiu, B. Li, W. Guo, and D. Song. The attack and defense landscape of agentic ai: A comprehensive survey. *arXiv preprint arXiv:2603.11088*, 2026.